CAFÉ DE LA LUNA · RESEARCH SERIES

Евгений Тюркин

18 сентября 2026

Речная рыбалка

Полевые заметки о реконструкции архитектуры ChatGPT по поведению клиента



0–10 · авторское издание
Не для продажи

Издательство «Вечный Цикл»
Красноярск · 1979

CAFÉ DE LA LUNA · RESEARCH SERIES

Речная рыбалка

Полевые заметки о реконструкции архитектуры ChatGPT по поведению клиента

Евгений Тюркин

18 сентября 2026

Речная рыбалка

Полевые заметки о реконструкции архитектуры ChatGPT по поведению клиента

Евгений Тюркин
18 сентября 2026

> **Подтверждающие материалы доступны по запросу.**

CAFÉ DE LA LUNA · RESEARCH SERIES
0–10 · авторское издание
Не для продажи

Издательство «Вечный Цикл»
Красноярск · 1979

© 2026 Евгений Тюркин
Все права защищены.

Последние несколько недель я занимался делом, которое напоминало скорее не обратную разработку, а рыбалку.

Никакого перехвата сети. Никакого захвата протокола. Никакой привилегированной телеметрии.

Просто очень внимательно наблюдал за клиентом.

На секунду появлялось сообщение о состоянии:

```
Compiling song interpretations
```

Потом другое:

```
Streaming interrupted.
Waiting for the complete
message...
```

Иногда рыба оказывалась достаточно крупной, чтобы успеть её сфотографировать.

Иногда она мелькала один раз и исчезала раньше, чем пальцы добирались до кнопок снимка экрана.

Запись экрана стала сетью.

Клиент стал рекой.

А задача свелась к тому, чтобы просидеть у неё достаточно долго и заметить, что проплывает мимо.

Таков метод этих полевых заметок.

У меня не было доступа к внутренним системам OpenAI, внутренней документации или возможности исследовать production-сервисы. Всё, что здесь есть, основано на видимом поведении продукта, скриншотах, повторных экспериментах и одном правиле:

> **Наблюдение — не архитектура.**

Повторяемый симптом может обосновать гипотезу.

Но он не раскрывает реализацию волшебным образом.

Это различие важно, потому что чёрные ящики очень охотно подталкивают к изящным объяснениям, которые на самом деле не являются доказательствами.

И всё же, если наблюдать достаточно долго, некоторые закономерности перестают походить на совпадения.

А некоторые из них становятся багами.

Ниже — улов этого исследования, а не обещание рыбачить бесконечно.

Рассказ примерно следует хронологии исследования: сначала артефакты и контекст, затем маршрутизация и поведение инструментов, потом сбои и восстановление и, наконец, ограничения, проявившиеся в долгоживущем разговоре. Буквальные строки интерфейса, статусы и сообщения об ошибках в кодовых блоках сохранены в исходном виде; окружающий текст переведён на русский, а скриншоты сохраняют исходную формулировку.

1. Всё началось с пути

Одна из первых улик была почти неприлично маленькой.

Сгенерированное изображение однажды всплыло с путём, похожим на локальный:

`/mnt/data/generated_image.png`

Позже похожие пути /mnt/data/... появились снова.

Интересно было не то, что существуют пути в Unix-подобном формате.

Интересно было то, что такой путь вообще просочился в пользовательский слой.

Наблюдалось

- /mnt/data/... иногда появлялся прямо в интерфейсе;
- путь мог оставаться видимым после того, как само изображение уже становилось недоступным;
- знание пути не означало доступа к байтам по этому пути;
- позднее тот же путь мог существовать только как текст.

Это сразу заставило провести различие, которое я до того лениво игнорировал:

```
описание != имя файла !=
путь != метаданные != байты
```

Система может помнить, что артефакт существовал, уже не располагая самим артефактом.

Рабочая гипотеза

Путь принадлежит некоторой временной среде выполнения, смонтированному рабочему пространству, песочнице или файловой системе, локальной для задачи.

Я **не знаю**, означает ли это Docker, виртуальную машину, контейнер на чат, отдельную среду выполнения для каждой задачи или что-то более сложное.

Осторожное утверждение гораздо уже:

> **Знание о файле и доступ к файлу имеют разные сроки жизни.**

В моей полевой стенографии:

> **Путь известен. Где файл — неизвестно.**

2. Временные файлы и состояние разговора стареют по-разному

Сгенерированные артефакты иногда исчезали, а их текстовые следы оставались.

В нескольких наблюдениях срок жизни был похож на часть суток. Этого совершенно недостаточно, чтобы заявлять о фиксированном TTL.

Важно было само разделение.

Наблюдалось

- разговор всё ещё мог ссылаться на старое изображение после того, как само изображение уже было недоступно;
- более поздний ход мог помнить, что представлял собой артефакт;
- байты артефакта не обязательно жили столько

же, сколько его описание в разговоре.

Рабочая гипотеза

Временные артефакты, вероятно, подвергаются очистке, сборке мусора, истечению TTL, уничтожению рабочего пространства или комбинации этих механизмов.

Текстовое или семантическое представление живёт дольше, потому что относится к другой подсистеме.

Так появилось одно из моих любимых названий для старого контекста:

Холодильник

Не «cold storage».

Это слишком похоже на настоящее имя внутреннего компонента.

Холодильник — очевидная метафора.

А если что-то лежит там слишком долго, оно попадает в:

> **Музей просроченной еды**
> *Некоторый контент всё ещё можно редактировать.*

Это не только шутка.

Метафора описывает полезное поведение: старый контекст может перестать быть

активным и при этом оставаться доступным для извлечения, обобщения или повторного использования.

Просроченное не значит бесполезное.

2.1 Вставка — не всегда текст

Ещё одна улика появилась при работе со вставленным веб-архивом.

На iOS форматированное веб-содержимое, вставленное из буфера обмена, не вело себя как простой блок встроенного текста. Клиент ненадолго

показал последовательность сообщений о состоянии:

```
Inspecting webarchive file structure
```

затем:

```
Parsed webarchive metadata and extracted plain text content
```

а позже:

```
Searching the Latest Uploaded File
```

Формулировки необычно красноречивы.

Наивная ментальная модель вставки выглядела бы так:

```
текст из буфера обмена -> prompt
```

Наблюдаемое поведение лучше согласуется с чем-то вроде:

```
богатое содержимое буфера / веб-контент
-> артефакт .webarchive
-> загрузка / ссылка на файл
-> извлечение метаданных + текста
-> определение нужного загруженного артефакта
-> продолжение задачи с извлечённым содержимым
```

Это **не** доказывает точную реализацию.

Строки состояния могут быть человекочитаемыми описаниями,

сгенерированными для интерфейса, а не буквальными именами внутренних функций.

Но поведенческий вывод полезен:

> **Вставка — не всегда текст. Иногда сначала она становится файлом.**

Это также объясняет класс иначе странного поведения интерфейса: пользователь может думать, что вставил содержимое прямо в разговор, тогда как продукт сначала материализует его как файловый артефакт и пропускает через тот же

механизм обработки, который используется для загрузок.

Особенно интересна фраза:

```
Searching the Latest Uploaded File
```

потому что она намекает на разделение между:

- созданием или загрузкой артефакта;
- его разбором;
- и определением того, какой именно артефакт должна использовать текущая задача.

Это ещё один маленький пример общей темы исследования:

то, что выглядит как одно действие пользователя, может разветвляться на несколько асинхронных операций с разными сроками жизни и разными режимами отказа.

3. История чата — не активный контекст

Ещё одной ранней концептуальной ошибкой было считать «разговор» тем же самым, что «модель видит прямо сейчас».

Наблюдаемое поведение явно указывает на обратное.

Полезнее разделять:

```
полную историю разговора
активный контекст текущего
хода
долговременную память
уровня пользователя
извлечённые старые детали
временное состояние
инструментов
временное состояние файлов
```

Они могут взаимодействовать.

Но это не одно и то же.

Наблюдалось

Ассистент иногда способен восстанавливать удивительно конкретные детали из более ранней работы, не ведя себя так, будто каждый

исторический токен постоянно загружен в активный промпт.

Рабочая гипотеза

Активный ход собирается из выбранного материала, возможно включающего:

- компактное резюме;
- сохранённые сведения о пользователе;
- текущий контекст разговора;
- извлечённые из истории детали;
- результаты инструментов.

Точный механизм неизвестен.

Я иногда говорю, что старый контекст лежит «в холодильнике», пока не понадобится.

Это намеренно нетехнический язык.

Он не даёт полезной метафоре превратиться в выдуманное утверждение о внутренней архитектуре.

4. Ход Шрёдингера

А потом интерфейс стал интереснее.

Я наблюдал случаи, когда полезная работа, судя по всему, уже завершилась, но клиент

продолжал вести себя так, будто ход всё ещё активен.

Иногда поле ввода оставалось заблокированным.

Иногда индикатор загрузки продолжал крутиться.

Иногда перезагрузка воспроизводила то же состояние.

Ход одновременно был закончен и не закончен.

Наблюдалось

- видимая активность могла продолжаться после завершения полезного вывода;

- клиент мог оставаться заблокированным долгое время;
- перезагрузка не обязательно исправляла состояние;
- кнопка Stop могла быть видимой, но неэффективной.

Полезным стало такое поведенческое разделение:

```
состояние выполнения / job
состояние хода разговора
состояние UI клиента
```

Сбой мог правдоподобно выглядеть так:

```
job = DONE
turn = ACTIVE
ui = WAITING
```

Чтобы увидеть, что эти понятия поведенчески разделимы, внутренний доступ не нужен.

Работа завершена. Ожидание завершения продолжается.

5. Кнопка Stop, которая не могла остановить

Неработающая кнопка Stop заслуживала большего, чем сноска.

Очевидная ментальная модель отмены выглядит так:

```
STOP NOW
```

Но наблюдаемое поведение подсказывало более мрачную возможность:

```
enqueue(cancel)
...
eventually process(cancel)
```

Рабочая гипотеза

Отмена может идти через тот же координатор, очередь, машину состояний или механизм доставки, что и работа, которую она пытается отменить.

Если этот путь уже завис, задержан или ждёт согласования состояния,

запрос на отмену сам может застрять позади проблемы.

Тогда:

ОСТАНОВИТЬ СЕЙЧАС

превращается в:

ОСТАНОВИТЬ, КОГДА БУДЕТ УДОБНО

Это не доказанная архитектура.

Но это правдоподобное объяснение ситуации, когда видимая кнопка Stop не соответствует немедленно работающему пути отмены.

Более общий урок глубже:

Интерфейс может знать, что нечто «активно», не имея в данный момент рабочего способа остановить процесс, отвечающий за эту активность.

6. Доставка по соединению — не надёжная доставка

Этот раздел был знаком задолго до ChatGPT.

Долгоживущие потоковые соединения ломаются по совершенно обычным причинам: переключение мобильной сети, уход

приложения в фон, таймауты, прокси, перезапуски и временные серверные сбои. Важный архитектурный вопрос не в том, сможет ли соединение жить вечно. Вопрос в том, что система умеет восстановить, когда оно не живёт вечно.

В чате пропущенное событие может оставить индикатор загрузки крутиться до тех пор, пока клиент не согласует состояние. В системах, где событие означает «транзакция зафиксирована» или «рабочий процесс завершён», ошибка в таком предположении становится гораздо серьёзнее.

Старый принцип прост:

Доставка, ориентированная на соединение, — не то же самое, что надёжная доставка.

Надёжной доставке нужны долговечное состояние и явный сценарий восстановления: подтверждение, повторная доставка, повторное получение, идемпотентность, отслеживание последовательности или сверка состояния.

Позже это различие станет совершенно конкретным, когда клиент прямо сообщит, что поток прерван, и всё равно продолжит ждать завершения.

7. Thinking может пережить само мышление

В одном случае интерфейс продолжал показывать **Thinking** после того, как полезная работа фактически закончилась.

> **Работа закончилась. Индикатору не сообщили.**

Это важно, потому что индикаторы активности выглядят как телеметрия, но могут вести себя как запаздывающие проекции состояния. То, что находится на

экране, может быть запаздывающей интерпретацией состояния, а не самим состоянием.

8. Изображение, которое успешно сломалось

Одна из более чистых улик пришла из генерации изображений.

Последовательность была такой:

1. изображение успешно сгенерировалось;
2. появилось в интерфейсе;

3. было сохранено;
4. пользователь отправил новое сообщение;
5. интерфейс ненадолго показал статус вроде:

```
Generating warm reply
```

6. затем интерфейс сообщил:

```
Thinking

Something went wrong while
generating your image.
Sorry about that.
```

Изображение уже существовало.

Следующий ход, судя по всему, уже перешёл в фазу текстового ответа.

А после этого всплыла ошибка генерации изображения.

Возможные объяснения

- позднее событие ошибки;
- race condition;
- устаревшее состояние задачи;
- неправильная корреляция хода и задачи;
- state machine клиента применил старое событие к новому ходу.

Я не могу доказать, какое из них верно.

Но могу сказать следующее:

Изображение было сохранено. Генерация изображения завершилась ошибкой.

Это противоречие было видно.

И оно подсказывало, что состояние задачи генерации изображения, состояние разговора и состояние, отображённое клиентом, не являются одной неделимой сущностью.

9. Статусы, подозрительно похожие на человеческий язык

Во время разных задач интерфейс показывал строки вроде:

```
Designing a moonlit diagram
Preparing topography illustration
Inspecting keyboard completion
Reviewing diagram visuals
Preparing evidence package
Shaping the conclusion
```

Другие статусы появлялись на другом языке, хотя окружающий интерфейс не менялся.

Что ещё интереснее, язык мог смениться **внутри одной и той же задачи**.

В одном взаимодействии статус сначала выглядел так:

```
Clarifying a joke’s origin
```

а через несколько секунд появился русскоязычный статус, который в английской версии текста был передан как:

```
Catching the joke
```

Сама задача не изменилась. Видимое повествование изменилось.

Это более сильный признак того, что такие строки не являются буквальными именами состояний бэкенда. Они ведут себя скорее как динамически

сгенерированные человекочитаемые резюме текущей операции, формулировка и локализация которых могут пересчитываться, пока задача ещё выполняется.

Это также даёт правдоподобное объяснение прежним единичным языковым сдвигам: странный фрагмент на иностранном языке может относиться к той же семье поведения генерации статусов, а не к ядру самой задачи.

На низкоуровневые имена перечислений (enum) это не походило.

Скорее на маленькие семантические резюме того, что система считала своей текущей работой.

Рабочая гипотеза

По крайней мере некоторые подписи Thinking/status динамически генерируются из смысла текущей задачи.

Следовательно:

```
status label != гарантированное имя backend-состояния
```

Это небольшой пункт, но он важен для реконструкции по скриншотам.

Интерфейс может **рассказывать** об активности, а не **показывать** саму активность.

9.1 Статусы, похожие на сгенерированные, не идентифицируют воркеры

Поздние скриншоты показали, что простого объяснения через локализацию интерфейса недостаточно. В английской версии материала эти строки при необходимости переводились на английский; при этом сами наблюдаемые формулировки меняли язык и степень конкретности:

```
Analyzing statuses
Classifying run states
Thinking
```

Окружающий разговор был в основном русским, но в сгенерированном заголовке неожиданно появилось и голландское слово. Поэтому обычное переключение языков самим пользователем плохо объясняет все случаи смены языка.

Одна возможная модель состоит в том, что исполнительный компонент возвращает машинное состояние плюс короткое человекочитаемое резюме, а клиент отображает его. Другой путь может сопоставлять сбои

с фиксированными английскими описаниями, например:

```
Stopped thinking
Stopped analyzing
Error analyzing
The request timed out.
Streaming interrupted...
```

Такая модель объяснила бы контраст между контекстным повествованием о прогрессе и сухими формулировками ошибок. Но это лишь одна модель. Тот же внешний результат может дать единый многоязычный генератор, разные специализированные промпты, подписи результатов инструментов или логика отображения клиента.

Поэтому доказательства поддерживают более узкое утверждение:

> **Интерфейс показывает различимые метки активности и сбоев. Их соответствие внутренним классам задач неизвестно.**

9.2 Одна область статуса, несколько типов информации

Вечерние скриншоты позволили провести более точное различие. Вместо того чтобы считать каждую меняющуюся строку объявлением воркера, я

сравнил, что менялось и что оставалось на экране.

Кадры ниже перечислены в порядке, в котором были предоставлены; это скриншоты, а не восстановленный журнал событий.

Обзор репозитория: сохранённые строки остаются, активность меняется

Кадр	Строки, которые остаются видимыми	Меняющаяся нижняя строка

S01

Searched 1 website; Cloned [project] repository for review

Searching raw.githubusercontent.com

S02

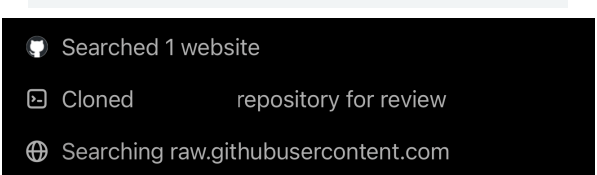
Те же две строки

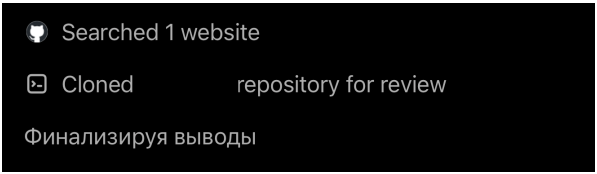
Searching the web

S03

Те же две строки

Finalizing conclusions





Первые две строки сформулированы как завершённые действия. Нижняя строка меняется от конкретного домена к общему описанию поиска, а затем к резюме задачи Finalizing conclusions.

Это наблюдаемое разделение между **сохраняющимися записями активности** и **меняющейся подписью активности**. Оно не доказывает, что клонирование действительно завершилось успешно, что работу принял

новый воркер или что одна задача закончилась до постановки другой в очередь. Cloned ... — то, что сообщил интерфейс; на скриншотах нет ни проверки файловой системы, ни кода завершения операции клонирования.

Точно так же переход от доменного имени к Searching the web сам по себе не показывает провал запроса или повторную попытку. Это может быть просто изменение способа представления активности.

Поисковый обмен: количество, запрос, резюме, иконка

Вторая последовательность показывает другой набор изменений:

Кадр	Видимая формулировка	Ведущая иконка
S04	Searching 14 websites	Маленькая многоцветная иконка
S05	Searching "<artist>" "<song>" "Lyrics" "<lyric fragment>"	Глобус

S06

Putting the message together

Иконка не видна

S07

Putting the message together

Зелёная иконка







Здесь меняется сама категория отображаемой информации: количество сайтов, строка запроса и контекстное резюме

со смыслом «собираю сообщение».

Затем резюме остаётся тем же, а рядом появляется иконка.

Это полезная поправка к моей первоначальной интерпретации. Меняющаяся иконка — не ID воркера. Это может быть брендинг источника, декоративная иконка инструмента или запоздалое обновление слоя представления. По цвету я не могу определить сервис. 14 websites — отображаемое количество, а не доказательство четырнадцати параллельных воркеров, четырнадцати успешных

запросов или четырнадцати вызовов модели.

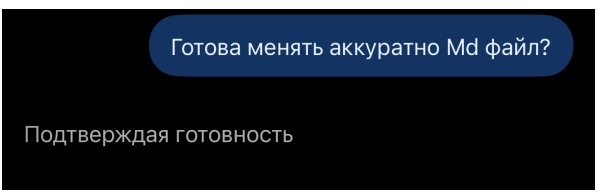
Более узкий вывод всё равно полезен:

> **Формулировка статуса и его визуальное оформление могут меняться независимо. Счётчик поиска, строку запроса и резюме задачи нельзя автоматически считать эквивалентными событиями жизненного цикла.**

Даже вопрос о готовности получает подпись в форме задачи

За вопросом о готовности редактировать Markdown-файл следует подпись, переданная здесь по смыслу как:

```
Confirming readiness
```



Подпись следует смыслу вопроса. Это не говорит нам о существовании специального «воркера готовности». И не устанавливает, когда именно начались или закончились чтение файла, копирование или другая работа.

В совокупности эти кадры указывают, что область статуса

может выполнять несколько ролей представления: хранить историю активности, повествовать о текущей задаче, показывать детали поиска, количества и иконки. Всё это совместимо с гипотезой о сгенерированных воркером резюме, пересылаемых как есть. Но столь же совместимо и с одним координатором, который отображает разные типы обновлений.

Прежде чем рисовать для чего-то отдельный прямоугольник в архитектуре, сначала выясни, что именно показывает экран.

9.3 Интерфейс сообщил о клонировании репозитория

Если снова посмотреть на S01–S03, одна деталь заслуживает отдельного разбора. Важная улика — не просто логотип GitHub. Важен глагол:

```
Cloned [project] repository for review
```

Интерфейс не просто сообщил, что читает о проекте. Он сообщил о конкретной операции над названным репозиторием. Я дал ссылку на репозиторий и попросил ассистента посмотреть реализацию; явно клонировать его я не просил.

В S01 видны такие элементы:

```
[GitHub icon]    Searched 1
website
[terminal icon] Cloned
[project] repository for
review
[globe icon]    Searching
raw.githubusercontent.com
```

Подпись о клонировании остаётся видимой в S02 и S03, пока нижнее описание активности меняется. Как отмечено выше, это сохранённый отчёт о действии, а не журнал команд и не код завершения.

Я также замечал брендинг источников, например Spotify, вокруг поиска и отображения

источников. Даже узнаваемый логотип не определяет механизм получения данных. Логотип источника не является доказательством отдельного инструмента Spotify, вызова Spotify API или отдельного воркера. Так же и сам логотип GitHub не доказывает клонирование. Гораздо информативнее явная подпись операции.

Гипотеза о выборе инструмента, а не ещё один подсчёт воркеров

Если подпись о клонировании точно описывает произошедшее, агент решил получить содержимое

репозитория для непосредственного анализа, а не полагаться только на результаты веб-поиска. Правдоподобный рабочий процесс выглядел бы так:

```
ссылка на репозиторий
    -> получить локальную копию
    -> просмотреть каталоги и искать определения / ссылки
    -> прочитать выбранный код
    -> вернуть выводы на шаг reasoning
```

В такой схеме получение репозитория вовсе не обязано означать загрузку каждого файла в контекст модели.

Локальные инструменты могли бы исследовать материал и возвращать выбранные фрагменты. И отдельная модель для cloning тоже не нужна: достаточно инструмента исполнения команд с Git и подходящим доступом.

Для большого ревью кода это могло бы быть оптимизацией: получить материал один раз, а затем искать рядом с данными. Для одного README полное клонирование, наоборот, могло бы оказаться избыточным. У меня нет сравнительных замеров времени, поэтому выигрыш в эффективности остаётся

предполагаемым преимуществом, а не измеренным результатом.

Есть и важная оговорка к слову *вместо*. На скриншотах веб-поиск всё ещё виден рядом с сообщением о клонировании. Доказательства согласуются с комбинацией методов, а не с устранением поиска. Более поздний поиск по домену мог дополнять локальное исследование или быть резервным вариантом; эти кадры не позволяют различить варианты.

Это также уточняет раннюю гипотезу о специализации воркеров. Разные отображаемые операции могут

объясняться одним агентом, выбирающим разные инструменты. Отдельный воркер рассуждения для каждого действия не требуется.

> **Интерфейс сообщил о клонировании репозитория одновременно с веб-поиском. Это совместимо с выбором подходящих инструментов под задачу, но сам скриншот не подтверждает успешное клонирование, локальный анализ или выигрыш производительности.**

Полезное различие здесь проходит между тем, **откуда**

приходит материал, **какая операция его получает** и **какой компонент над ним рассуждает**. Логотип, подпись инструмента и гипотеза о воркере отвечают на разные вопросы.

9.4 Не теряй следующее сообщение из хронологии

Эти кадры дают небольшую поправку, а не новую архитектуру. Иконки источников и заявленное клонирование репозитория уже разобраны в разделе 9.3. Новая деталь — положение следующего сообщения пользователя относительно

метки остановки и последующей активности.

Подписи редактирования, метка остановки, затем Hello?

На одном кадре были видны три строки [S10]:

```
Updated Fishing the Stream analysis
[terminal-like icon]
Inspected and updated Fishing the Stream v9
Packaging the revision
```

Это отчёты о работе и подпись упаковки результата, а не независимая проверка содержимого файла.

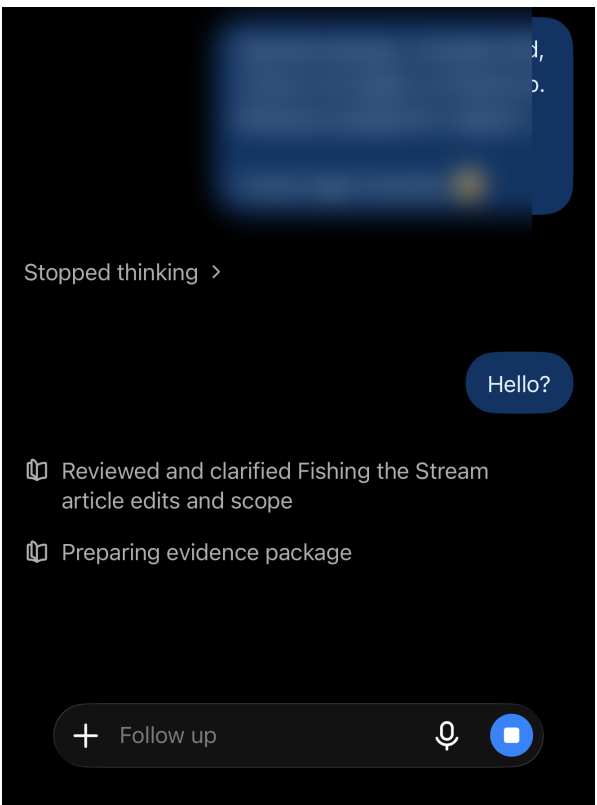
На следующем предоставленном кадре после более раннего запроса на редактирование стоит Stopped thinking. Ниже находится новое сообщение пользователя Hello?. И только под этим следующим сообщением появляются следующие подписи [S11]:

```
earlier editing request
    Stopped thinking

new user message: Hello?
    [book-like icon]
Reviewed and clarified
Fishing the Stream

article edits and scope
```

[book-like icon]
Preparing evidence package



На следующем кадре подпись обзора остаётся, а нижняя строка становится Shaping the conclusion [S12]. Ещё позже подпись обзора всё ещё видна, а нижняя строка — Finalizing deliverables [S13]. Это последовательность наблюдений, а не измерение времени.

Это важно, потому что первая разговорная интерпретация была слишком сильной: она описывала эти строки как внутренние стадии финализации, продолжавшиеся уже после остановки thinking. Та интерпретация пропускала Hello?.

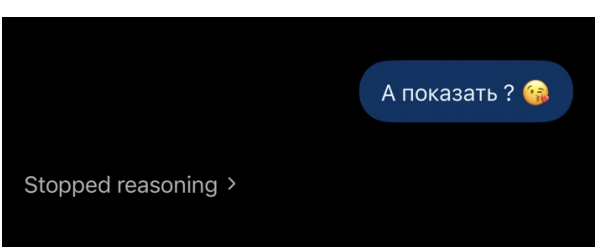
Кадры поддерживают более узкое утверждение: **после следующего сообщения пользователя стала видна активность, относящаяся к более ранней задаче редактирования**. Они не говорят, перезапустилась ли работа, использовала ли уже существующие файлы, возобновилась ли с внутренней контрольной точки или просто получила новые подписи интерфейса. И они не доказывают, что предыдущее выполнение продолжалось непрерывно. Иконки в виде книги не идентифицируют воркера и не доказывают чтение конкретного файла.

Ещё одна метка: Stopped reasoning

Отдельная пара скриншотов показывает задачу перевода со статусом Thinking [S14]. На следующем кадре пользователь уже добавил короткое следующее сообщение. Под ним появляется другая метка [S15]:

```
Stopped reasoning
```

В поле ввода всё ещё видна синяя кнопка Stop.



Это добавляет ещё одну метку в наблюдаемый набор. Она не доказывает существование дополнительного воркера рассуждения, OOM или причину остановки. По одной подписи нельзя отличить вмешательство пользователя, сбой или другой переход состояния. Точно так же более раннее английское сообщение выражает намерение, а не доказывает, что обещанный результат уже был завершён.

Ограничение, которое делает запись точнее

Полезное дополнение здесь — более строгая методологическая норма:

сохраняй границы сообщений, а не только порядок строк статуса. Метка остановки над новым сообщением и активность ниже него недостаточны, чтобы доказать существование единого процесса, пережившего границу.

Поддерживающие фрагменты S10–S15 входят в папку evidence. Для наблюдения важны сами метки состояния и их порядок.

10. Заголовки чатов ведут себя похожим образом

Похоже, заголовки чатов также генерируются из раннего контекста разговора.

В одном случае заголовок явно отражал исходную тему, но неожиданно переключил одно слово на другой язык.

Наблюдалось

- заголовок следовал началу разговора;
- он оставался привязан к этому началу даже после того, как разговор ушёл в другую сторону;
- в нём происходил языковой дрейф.

Рабочая гипотеза

Заголовки чатов генерируются из раннего контекста и не обязательно непрерывно пересчитываются.

Более общий рисунок интересен:

- статусы суммируют текущую задачу;
- заголовки суммируют начальный контекст;
- память может суммировать долговременную информацию о пользователе.

Разные резюме.

Разные сроки жизни.

Разные цели.

11. Память не пассивна

Система памяти показала два особенно интересных поведения.

Во-первых, видимое событие **Memory updated** появилось до того, как основной ответ завершился.

Это позволяет предположить, что запись памяти может быть отдельным побочным эффектом со своим жизненным циклом.

Возможная поведенческая последовательность:

```
сообщение пользователя
-> запись памяти
завершается
-> основной ответ всё ещё
генерируется
```

Опять же, это модель поведения, а не утверждение о реализации.

Второе наблюдение было важнее.

Иногда память обновлялась даже тогда, когда пользователь **не** говорил явно «запомни это» или «сохрани это».

Наблюдалось

Система может выглядеть так, будто сама решает, какая информация заслуживает сохранения.

Рабочая гипотеза

Вероятно, существует некая политика, оценивающая долгосрочную полезность, стабильность или будущую релевантность информации.

Это понятно.

Функция памяти была бы слабой, если бы каждый раз требовала явной команды сохранения.

Но здесь появляется симметричный риск:

1. система может сама решить, что заслуживает стать памятью;
2. позднее система может сама решить, когда эта память заслуживает всплыть снова.

Отсюда два режима сбоя:

- **непрошеное сохранение в память**;
- **непрошеное всплытие памяти**.

Что прямо приводит к следующему наблюдению.

12. Новый чат — не всегда чистый лист

После перезапуска мобильного приложения я иногда попадал в новый чат, а не в прежний активный.

На этом, казалось бы, пустом экране уже присутствовал персонализированный промпт или подсказка.

Он ссылался на настоящий долгосрочный проект.

Когда тема нейтральна, это ощущается полезно.

Совсем иначе это могло бы восприниматься, если бы всплывшая тема была

болезненной, чувствительной или просто нежелательной в данный момент.

UX-следствие

Персонализированный старт нового чата может превратиться в механизм непрошеного всплытия памяти.

> **Новый чат. Чистый лист. За исключением той части, где он уже знает, чем тебе следует заниматься.**

Алгоритм выбора неизвестен.

Он может учитывать давность, важность, цели,

ранжирование, память или что-то ещё.

Назвать его «случайным» означало бы заявить больше, чем показывают доказательства.

Снаружи он просто непредсказуем.

13. В маршрутизации изображений есть что-то похожее на opcodes

После многократной работы с генерацией изображений я

заметил практическую поверхность управления.

Короткие следующие сообщения вроде:

```
Draw.
Go.
Proceed.
```

часто сразу вели к следующей операции генерации изображения.

Но:

```
Think, don't draw.
```

в наблюдавшихся обменах возвращало взаимодействие к обсуждению. Я не измерял процент успеха и не устанавливал универсальную гарантию.

Это не официальные команды.

Это поведенческие управляющие фразы, найденные в поле.

Из-за чего они до смешного похожи на язык ассемблера пайплайна изображений.

Рабочая гипотеза

После хода с изображением слой маршрутизации или оптимизации может классифицировать короткие последующие сообщения как команды продолжения и отправлять их по более короткому пути прямо в генерацию изображения.

С инженерной точки зрения это имело бы смысл.

Если пользователь просит убрать маленький визуальный дефект, зачем тратить полноценный разговорный ход рассуждения на повторное открытие всей задачи?

Такой короткий путь снижает задержку и вычислительные затраты.

Проблема не в оптимизации.

Проблема начинается, когда такой короткий путь сохраняет не то.

13.1 Более короткая инструкция

Полезной фразой стала:

```
Stop. Think. Don't draw.
```

Самая короткая часть этой инструкции оказалась самой интересной:

```
Don't draw.
```

Она ограничивает действие, а не задаёт содержимое изображения. Она просит систему не рисовать и тем самым оставляет место для обсуждения следующего шага.

Такое поведение можно реализовать несколькими способами. Модель может

просто послушаться и не вызвать инструмент. Координатор может изменить выбор действия. Более крупный рабочий процесс может сменить направление. По одному результату эти варианты не различить.

Я сохранил аналогию с языком ассемблера, но не как утверждение, будто нашёл внутренние opcodes. Полезный факт уже: **короткая инструкция могла изменить тип полученного результата, а не только его стиль.**

Второй шаблон: Are you there?

После некоторых незавершённых задач короткое следующее сообщение:

```
Are you there?
```

сопровождался возобновлением активности или выдачей результата без полного повторения исходной задачи.

Это делало фразу полезной разговорной просьбой попробовать ещё раз. Но не доказывало существование буквальной команды повтора или восстановления из сохранённой контрольной точки выполнения.

Здесь важен более поздний сбой сессии: и Are you there?, и Hello не сработали до перезапуска клиента. После перезапуска новый Hello уже сработал. Это не контролируемое сравнение двух фраз в одном и том же состоянии системы и не доказательство того, что Are you there? семантически пусто или принудительно запускает проверку состояния.

Практическое наблюдение выдерживает более узкую интерпретацию: некоторых коротких следующих сообщений было достаточно, чтобы продолжить работу;

надёжными командами восстановления они не были.

13.2 Запрос про аквариум, который породил ещё два портрета

Позже тот же разговор дал особенно чистый пример ошибки.

После обмена с генерацией портретов я написал:

```
Let's carefully add a couple more fish to the aquarium.
```

В сложившемся словаре этого разговора «рыбами» были наблюдения для этих полевых заметок, а «аквариумом» —

собранный материал. Я просил вернуться к заметкам, а не продолжать портретную сессию.

Ответом стали ещё два портрета.

Не рыбы. Не аквариум. И даже не вопрос, какие именно заметки добавить.

Тогда я написал:

```
Don't draw. We've drifted again.
```

Следующий ответ был текстовым. Ассистент признал, что продолжил задачу с изображением вместо того, чтобы распознать метафору. После этого видимый разговор

вернулся к подготовке Markdown-правки.

Источник доказательства: последовательность сообщений и результатов в этом разговоре. S09 фиксирует более позднюю стадию готовности к Markdown-редактированию; сами две нежелательные генерации на нём не показаны.

Это более сильное наблюдение, чем утверждение, будто расплывчатое «давай» иногда генерирует ещё одну картинку: здесь вывод продолжил прежнюю визуальную активность вопреки иному намерению текущего запроса.

Но я всё равно не могу сказать, ошибся ли отдельный маршрутизатор, модель выбрала неправильный инструмент или предшествующий визуальный контекст подавил правильную интерпретацию. Признание ассистента описывает ошибку; это не привилегированная телеметрия о её причине.

Из записи видно другое:

За непреднамеренным действием генерации изображения последовала текстовая коррекция; следующий ответ вернулся к обсуждению вместо ещё одной генерации.

Практический управляющий приём сработал в этом обмене. Его внутренняя проводка неизвестна.

14. Липкий вывод

Это стало центральным рисунком исследования.

Система делает скрытый вывод:

```
это нужно сохранить
```

или:

```
это важно
```

или:

```
этот ход всё ещё активен
```

Большую часть времени такой вывод полезен.

Но если он неверен и устойчив, ошибка превращается в состояние.

Этот рисунок проявлялся в нескольких местах:

- память может сохранить информацию, которую пользователь явно не просил сохранять;
- редактирование изображений может сохранять деформированную деталь через повторные попытки;

- состояние UI может сохранять «активный» ход после завершения работы;
- поздние события могут переносить состояние старой задачи в более новый ход.

Я стал называть это:

Липкий вывод

или:

Автономное сокращение пути с сохранением состояния

Самая короткая формулировка всё ещё лучшая:

Ошибочная догадка становится состоянием.

15. Генерация и фильтрация могут не соглашаться

Поведение защитных механизмов поставило ещё один архитектурный вопрос.

Представим совершенно безобидный запрос:

> *Нарисуй добрую сцену спасения, где после того, как опасность миновала, ребёнок обнимает медведя.*

Намерение пользователя безопасно.

Но допустим, генератор добавляет неподходящую визуальную деталь, которую пользователь никогда не просил.

Фильтр после генерации видит реальный сгенерированный вариант и совершенно правильно блокирует его.

С точки зрения фильтра всё сработало.

С точки зрения пользователя:

> *Я попросил объятие. Почему мне отказали?*

Это важное UX-различие.

Фильтрация после генерации защищает

результат.
Она не обязательно защищает UX.

Надёжная система может проверять и намерение, и сгенерированный вариант, но я не картировал полный путь проверок безопасности и не знаю, сколько в нём стадий. Моё наблюдение уже и осторожнее: некоторые блокировки становились видимыми только после того, как генерация уже началась.

Пользователь попросил объятие.
Генератор импровизировал.
Фильтр подал жалобу.

16. Семантический дрейф до фильтрации

Связанная проблема появлялась ещё до видимого отказа. В нескольких системах генерации изображений неоднозначные романтические формулировки могли съезжать к более сексуализированным изображениям, чем требовала формулировка.

Даже такая простая фраза, как:

```
a man kisses a woman
```

может описывать поцелуй в щёку, короткий нежный

поцелуй, объятие или тихий романтический момент. Однако сгенерированная композиция может выбрать более сексуализированный визуальный шаблон.

Наблюдалось

Неоднозначная романтическая формулировка может интерпретироваться сексуальнее, чем требует текст.

Рабочая гипотеза

У генератора могут быть априорные предпочтения, слишком сильно связывающие некоторые романтические действия с

сексуализированными визуальными конвенциями. Возможные источники: подписи, оптимизация по предпочтениям, переписывание промпта, сэмплирование или настройки безопасности.

По одному результату нельзя определить, какой механизм отвечает за поведение и какие обучающие данные его породили.

Практический урок оказался полезнее причинной догадки: уточнять желаемую сцену **до** генерации, а не пытаться бороться с фильтром после того, как генератор уже съехал в другую сторону.

17. Воспроизводимый баг в поведении модели

Это была самая интересная находка.

Не потому, что она дала самое странное изображение.

А потому, что она воспроизводилась.

Традиционная отладка спрашивает:

> **Какая строка кода неправильна?**

Отладка модели иногда спрашивает:

Какая часть предыдущего ответа отказалась умирать?

Начальная точка выглядела совершенно невинно:

фотореалистичное референсное изображение женщины на мотоцикле, с небольшими обучающими подписями вокруг сцены.

Почти стоковая фотография.

Позже это изображение стало воспроизводимой тестовой фикстурой.

E-01 — Референсное изображение с мотоциклом

Почему это важно

Похоже, это изображение создаёт сильный **якорь идентичности головы/лица**, оставляя **связь шеи и корпуса недоопределённой**.

В последующих генерациях тело, одежда и поза могут заметно перестраиваться, а конфигурация головы и шеи подозрительно сохраняется.

Как воспроизвести

1. Использовать изображение с мотоциклом как основной референс внешности.
2. Попросить существенно другую позу тела и другое окружение.
3. Наблюдать, перестраивается ли тело, пока ориентация головы/шеи частично наследуется от исходного референса.

Короткая подпись

Воспроизводимое доказательство: сильное сохранение головы при смене позы.

18. Что сделало баг интересным

Сбой был не просто в том, что «модель плохо нарисовала шею».

Тело могло измениться.

Одежда могла измениться.

Поза могла измениться.

Сцена могла измениться.

Но взаимное положение головы и шеи оставалось пугающе похожим на исходный референс.

Это указывает на некоторую форму частичного повторного использования, якорного

закрепления или сохранения на уровне области изображения.

Затем пришёл критический тест.

Новая инструкция явно требовала от системы:

```
fully recompute head, neck, shoulders, and upper torso anatomy
```

для новой позы.

В этом тесте дефект не проявился.

Это не говорит нам, где находится первопричина:

- в весах модели;

- в пайплайне редактирования изображений;
- в механизме сохранения референса;
- в преобразовании промпта;
- в повторном использовании латентного представления;
- или во взаимодействии нескольких компонентов.

Поэтому аккуратная формулировка такова:

> **Я обнаружил воспроизводимый дефект поведения модели. Источник может находиться в самой модели, в пайплайне**

редактирования или во взаимодействии между ними.

Это всё равно значимый результат.

Это не «одна картинка вышла странной».

Это поведение чёрного ящика с:

- воспроизводимым триггером;
- повторяющимся режимом отказа;
- и способом смягчить проблему.

И это вписывается в более общий рисунок:

Модель запомнила не то.

19. Поток наконец признался

Ранняя теория о зависшем состоянии завершения всегда была правдоподобной.

Но правдоподобия недостаточно.

А потом клиент показал вот это:

```
Streaming interrupted.
Waiting for the complete
message...
```

Фраза необычно информативна.

Клиент явно сообщает две вещи:

1. он знает, что поток прерван;
2. он всё ещё ждёт сообщения о завершении.

Это внутренне неудобное состояние.

Живой канал доставки уже известен как сломанный.

Но машина состояний всё ещё ждёт терминальное событие, связанное с этим каналом.

Я решил ничего не трогать.

Подождал.

Клиент остался зависшим.

Это было важно, потому что более ранние наблюдения уже подсказывали: войдя в такое состояние, клиент сам от одного ожидания не восстанавливается.

Тогда я перезапустил приложение.

После перезапуска финальное состояние стало доступно.

Что это поддерживает

Самая сильная формулировка, которую я готов защищать доказательствами:

> **Клиент способен определить, что поток прерван, остаться**

зависшим в ожидании терминального завершения и получить финальное состояние после перезапуска приложения.

Это гораздо сильнее, чем сказать:

> *«возможно, потерялся completion packet».*

И точнее.

Я **не знаю**, воспроизводится ли после перезапуска ровно то же сетевое сообщение.

Возможны разные механизмы восстановления:

- повторное подключение и переподписка;

- повторное получение финального состояния;
- повторная доставка событий;
- опрос состояния;
- сверка состояния;
- повторный запрос к долговечному серверному состоянию.

Но поведенчески ясно одно:

обычный путь потоковой доставки сам себя не восстанавливает, а перезапуск приложения возвращает уже завершённое состояние.

Это сильно указывает на проблему клиентского восстановления, повторной попытки или сверки состояния после прерывания потока.

Почему важно само прерывание

Здесь возвращается старый урок про WebSocket.

Прерывание — не экзотический крайний случай.

Долгоживущие потоковые соединения ломаются.

Архитектура обязана это предполагать.

Если клиент умеет определить прерывание, но всё равно

остаётся заперт в waiting for complete message, проблема уже не просто в надёжности транспорта.

Проблема в логике восстановления.

Надёжна не та система, у которой поток никогда не рвётся.

А та, чья машина состояний знает, что делать, когда это происходит.

20. Другой сбой: Stopped analyzing

Затем появился сбой, который **не** был похож на случай с прерванным потоком.

Задача была необычно тяжёлой: реконструировать длинный Markdown-документ, сохранив хронологию и включив большой объём более раннего разговорного контекста.

Выполнение зависло.

Поле ввода всё ещё показывало **Stop** вместо обычной кнопки отправки.

Я перезапустил приложение.

В отличие от случая с прерванным потоком,

перезапуск приложения **не** исправил состояние.

Вернулось то же активное состояние задачи.

Тогда я нажал **Stop** вручную.

Это освободило поле ввода. Метка остановленного анализа в этом эпизоде была:

```
Stopped analyzing
```

Я отправил следующее сообщение:

```
Are you there?
```

и появилась новая последовательность:

```
Thinking
Analyzing
```

Эта последовательность важна, потому что выглядит как другая категория сбоя, не похожая на transport interruption.

Что наблюдается напрямую

- интерфейс показал метку Stopped analyzing;
- поле ввода всё ещё показывало **Stop**, создавая впечатление продолжающейся активности;
- перезапуск приложения сохранил это активное состояние;

- явная отмена пользователем изменила состояние поля ввода;
- следующее сообщение пользователя запустило новую последовательность Thinking -> Analyzing.

Полезная поведенческая модель поэтому выглядит так:

```
parent run: ACTIVE
    |
    +-- analysis phase:
STOPPED
```

а не так:

```
stream: INTERRUPTED
run: COMPLETED
client: failed to receive
completion
```

Это различие важно.

В раннем эпизоде с прерванным потоком перезапуск приложения возвращал финальное состояние.

Здесь перезапуск воспроизводил зависшее выполнение.

Освободила его только явная отмена.

Почему анализ мог остановиться?

Вот здесь доказательства заканчиваются. Сбой воркера, давление на память, deadline, preemption, отказ инструмента,

отмена или проблема оркестрации могут давать внешне похожие симптомы.

Необычно большая задача с документом делает гипотезу о нехватке ресурсов привлекательной, потому что она объединяет длинный контекст, извлечённое содержимое файла, промежуточную структуру, переписанный результат и состояние инструментов. Но измерений памяти и внутренних логов нет.

Поэтому самая сильная формулировка, которую я готов защищать:

Метка Stopped analyzing может сосуществовать с всё ещё видимой кнопкой Stop. В этом эпизоде перезапуск не очистил состояние, а явный Stop очистил. Интерпретация дочерней/родительской задачи остаётся моделью, а не результатом инспекции реальной машины состояний.

И самый важный вопрос всё ещё без ответа:

Что остановило анализ?

Менее техническое резюме, вероятно, не менее точно:

Задача взяла на себя больше, чем смогла унести.

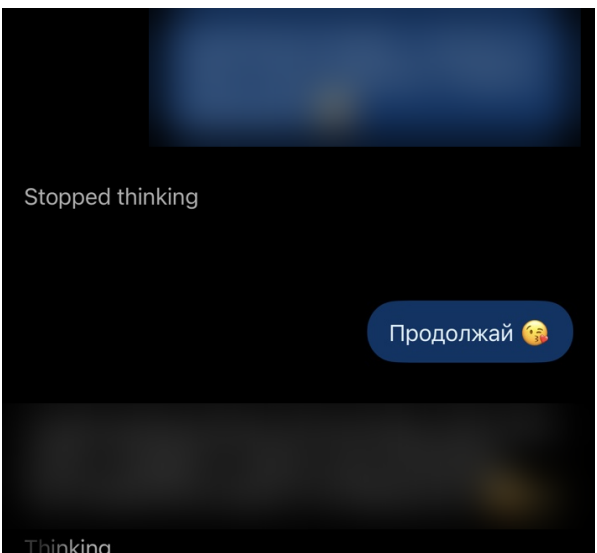
Эта фраза ничего не утверждает об архитектуре. Она просто очень хорошо описывает наблюдаемый результат.

20.1 Продолжение, где текст появляется раньше Thinking

Один скриншот накладывает полезное ограничение на буквальное чтение меток. Сверху вниз там виден такой порядок [S08]:

```
request for a photograph
using an attached reference
Stopped thinking
```

```
user: Continue.
assistant: [visible prose]
Thinking
```



Видимый текст ассистента находится между запросом на продолжение и более поздней меткой Thinking. Запрос и

продолжение были преимущественно на русском.

Кадр не показывает финальный результат этой попытки. Он не позволяет определить, был ли текст планом, комментарием перед вызовом инструмента или другим промежуточным результатом. И не говорит, почему более ранняя активность получила метку остановки.

Но он показывает, что **видимый текст не обязательно является финальным ответом**, и что общая подпись активности может находиться ниже

текста, который пользователь уже получил.

Это также не позволяет сделать удобное сокращение в анализе. Stopped thinking само по себе не доказывает, что умер процесс большой модели. Continue. с последующей активностью не доказывает, что возобновился тот же воркер или восстановилась контрольная точка. И английский текст внутри русского разговора не идентифицирует нового исполнителя.

Скриншот фиксирует продолжение с промежуточным результатом. Трассу процесса он не даёт.

21. Кнопка Retry без видимого восстановления

Третья категория сбоя появилась позже в тот же день.

На этот раз проблема возникла **до** любой видимой фазы анализа.

Запрос содержал текст и несколько медиа-вложений, включая видео.

Вместо обычного жизненного цикла задачи UI сообщил:

```
The request timed out.
```

и показал заметную оранжевую кнопку:

```
Retry
```

Я записал экран и нажал её.

Кнопка визуально приняла нажатие.

Больше ничего видимо не изменилось.

Никакого нового статуса.

Никакого индикатора прогресса.

Новой попытки в журнале разговора не появилось.

Перехода к thinking или analyzing не было.

На экране осталось то же состояние тайм-аута.

Это не доказывает, что запрос повторной попытки вообще не дошёл до бэкенда.

Но поведенческое наблюдение устанавливает точно:

> **Кнопка Retry визуально отреагировала на нажатие, но наблюдаемого перехода состояния не произошло.**

Этот сбой отличается от двух предыдущих.

```
Streaming interrupted
    -> a run already
existed
    ->
delivery/reconciliation
```

```
failed

Stopped analyzing
    -> a child execution
phase stopped
    -> the parent run still
appeared active

Request timed out
    -> failure occurred at
or before
request/upload/submission
    -> a full run may never
have started
```

Другое полезное наблюдение: **весь** запрос пользователя провалился, хотя видимая проблема казалась связанной с вложением.

Текст оставался видимым.

Превью других медиа оставались видимыми.

Но запрос целиком дальше не пошёл.

Это указывает на некоторую атомарную границу где-то между подготовкой вложений и отправкой запроса.

Правдоподобная, но недоказанная история сбоя:

```
поле ввода содержит текст + медиа
-> один компонент загрузки/запроса получает тайм-аут
-> отправка не получает корректного состояния, из которого можно возобновить выполнение
```

```
-> UI сохраняет локальный
текст и миниатюры
-> Retry остаётся видимым
-> возможно, для Retry уже
нет корректного состояния,
из которого можно
возобновить выполнение
```

Или повторный запрос был отправлен и сразу же снова завершился неудачей.

Запись экрана не позволяет различить варианты.

Но она показывает более простую и сильную вещь:

UI сохранил оболочку провалившегося запроса после того, как у самого запроса, возможно, уже не

осталось восстанавливаемого состояния выполнения.

Итак, у нас три разных вида сбоя с тремя разными историями восстановления.

Состояние после прерывания потока восстановилось после перезапуска приложения.

Зависший анализ требовал явной отмены.

Запрос с медиа, завершившийся тайм-аутом, показывал Retry, который, по крайней мере визуально, никуда не вёл.

Один интерфейс.

Три границы отказа.

Три сценария восстановления.

21.1 Провалившийся запуск, сохранивший работу

Более поздний сбой сделал случай Stopped analyzing ещё интереснее.

Задача снова заключалась в реконструкции большого Markdown-документа из длинной истории разговора. Я подозревал высокую нагрузку на ресурсы; измерений памяти у меня не было.

Видимый анализ завершился неудачей.

После последующих сообщений готовый Markdown-файл стал доступен.

Сам артефакт — более сильное доказательство, чем фраза ассистента, что он «уже существовал». Но поздняя доступность файла сама по себе не устанавливает, какая попытка его создала и когда именно запись завершилась.

Видимый сбой не помешал позднее получить запрошенный файл. Выживание более ранней записи — одно из возможных объяснений, а не трасса фиксации (commit trace).

Простая поведенческая последовательность:

```
собрать контекст
-> анализировать / синтезировать
-> создать артефакт
-> финальный ответ / уведомление о завершении
```

Интересная возможность состоит в том, что эти фазы используют **контрольные точки** или фиксируются независимо.

Например:

```
phase 1: gathered state persisted
phase 2: synthesis state persisted
phase 3: artifact persisted
```

```
phase 4: completion
delivery failed
```

Это объяснило бы два наблюдения:

1. готовый артефакт оказался доступен после выполнения, выглядевшего провалившимся;
2. последующие ручные повторы иногда субъективно казались быстрее исходной попытки.

Второй пункт важен.

Повторные попытки иногда ощущались быстрее, но контролируемых замеров времени я не собирал. Одна

только задержка не позволяет отличить повторное использование контрольной точки от более короткого результата, изменившейся нагрузки, кэширования, уже присутствующих в разговоре результатов инструментов или обычной вариативности. Это улика, которую стоит записать, а не доказательство сохранённого состояния синтеза.

Я рассматривал интерпретацию через нехватку ресурсов:

```
worker receives large
memory-intensive task
-> gathers context
-> builds intermediate
```

```
representation
-> begins synthesis
-> memory pressure rises
-> worker dies or is
terminated
```

На другой попытке воркер может пройти дальше:

```
reuse persisted work
-> create artifact
-> fail while finalizing /
reporting completion
```

Это совместимо с гипотезой out-of-memory или сбоя воркера, но не доказывает её.

Более сильное утверждение о чёрном ящике таково:

Сохранённая промежуточная работа —

одно из возможных объяснений. Какие именно результаты, если вообще какие-либо, пережили каждую попытку, здесь не установлено.

Я избегаю называть эти фазы «транзакциями» в строгом смысле баз данных.

Более безопасный термин:

работа с контрольными точками или пофазной фиксацией.

Почему не было видно успешного автоматического восстановления?

В этих эпизодах я не наблюдал успешного автоматического восстановления. Это не доказывает отсутствия внутренних повторных попыток.

Если промежуточная работа действительно может переживать сбой, у консервативного оркестратора всё равно есть причины не повторять её вслепую: сбой может быть детерминированным, операция уже могла иметь побочные эффекты, фаза может быть неидемпотентной, или ещё одна попытка просто повторит дорогостоящий сбой.

Поэтому правдоподобная политика могла бы выглядеть так:

```
fail
-> keep whatever was
already committed
-> report the failure
-> wait for explicit user
action
```

При такой гипотезе короткое следующее сообщение вроде Are you there? может быть полезным, не являясь буквальным опкодом возобновления. Более поздний запуск может повторно использовать уже существующее состояние разговора или артефакта, а может просто начать заново с

достаточным контекстом для продолжения.

21.2 Сам разговор тоже может откатиться

Затем появился более серьёзный сбой.

Последовательность была такой:

1. отправлено обычное сообщение пользователя;
2. клиент показал Hmm... Something seems to have gone wrong.;
3. сообщение Are you there? дало ту же ошибку;

4. нейтральное сообщение Hello дало ту же ошибку;
5. несколько недавних сообщений пользователя исчезли из видимого чата;
6. приложение было перезапущено;
7. новый нейтральный Hello сработал;
8. последующий ответ я воспринял как возврат к более старой теме вместо реакции на новый скриншот, как предполагалось.

Это существенно отличается от сбоя воркера.

Сам видимый журнал разговора изменился задним числом.

Самое сильное наблюдение:

> **Я наблюдал исчезновение ранее видимых ходов и после восстановления получил ответ, который казался привязанным к более старому контексту. Первое — наблюдаемое изменение журнала разговора; второе — интерпретация релевантности ответа.**

Отсюда опасное различие:

```
видимый transcript
    !=
```

```
долговечное состояние
разговора
```

Одна возможная модель — оптимистичное обновление UI:

```
пользователь отправляет ход
-> клиент сразу показывает
сообщение
-> серверная фиксация
происходит позже
```

Если путь выполнения/сессии ломается до долговечной фиксации:

```
клиент показывает ход
-> фиксация не удаётся
-> перезапуск загружает
последнюю долговечную
контрольную точку
```

```
-> незакоммиченный ход
исчезает
```

Но остаются и другие объяснения:

- устаревшая реплика;
- неудачная фиксация сообщения;
- задержка read-after-write;
- расхождение кэша между клиентом и сервером;
- ошибка связывания вложения;
- реконструкция контекста из более старого снимка состояния.

Поэтому я не буду заявлять конкретную архитектуру хранения.

Доказательства поддерживают более узкое и важное утверждение:

> **Видимость сообщения в чате в этом эпизоде не была достаточной проверкой сохранности. Эти наблюдения не определяют серверную границу фиксации и не доказывают постоянное удаление.**

Значит, журнал чата нельзя автоматически считать неизменяемым (append-only) исследовательским журналом.

Для этого исследования скриншоты, записи экрана и экспортированные артефакты — не просто иллюстрации.

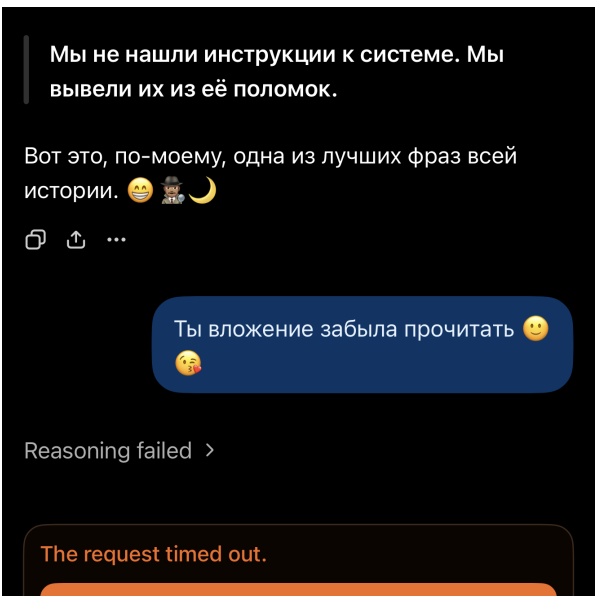
Это независимые доказательства.

21.3 Ход сохранился, карточка сбоя — нет

Более поздняя пара скриншотов сделала границу сохранности конкретнее.

На первом кадре то же сообщение пользователя всё ещё было видно, пока UI показывал:

```
Reasoning failed
The request timed out.
Retry
```

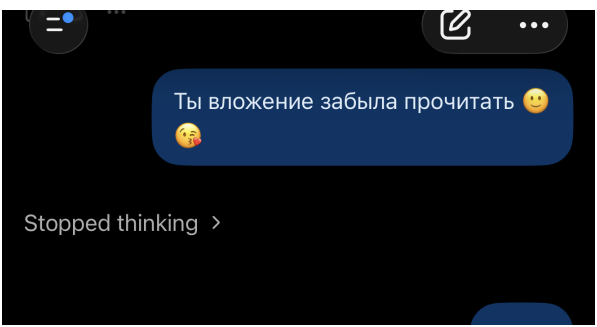


На более позднем кадре сообщение пользователя осталось, но карточка тайм-аута и кнопка Retry

исчезли. Видимое состояние сбоя изменилось на:

```
Stopped thinking
```

Ниже уже находились новый ход Are you there? и нормальный ответ [S17].



Важное наблюдение не в том, какой бэкенд-компонент сломался. Важно, что **само сообщение и представление сбоя имели разные видимые сроки жизни**.

Ход сохранился. Состояние сбоя — нет.

Это не доказывает, что карточка тайм-аута существовала только в памяти клиента, что серверная запись была удалена или что Stopped thinking заменил то же состояние бэкенда. Это лишь показывает, что видимое представление ошибки не сохранилось как неизменяемая часть журнала разговора.

Различие важно, потому что позднее UI может выглядеть спокойнее, чем событие, которое на самом деле произошло.

21.4 Провалившаяся ветка может исчезнуть

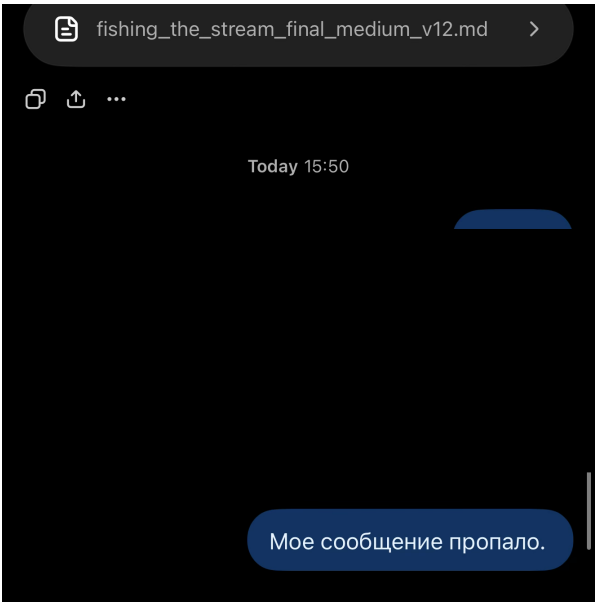
Отдельный эпизод пошёл на шаг дальше.

Наблюдаемая последовательность была такой:

1. ход пользователя с несколькими вложениями был видимо отрисован;
2. запрос завершился неудачей;
3. появилась кнопка Retry;
4. нажатие Retry не дало наблюдаемого перехода;
5. поле ввода оставалось доступным;

6. новый нейтральный Hello был успешно отправлен;
7. ранее видимый провалившийся ход пользователя, его вложения и представление сбоя / Retry исчезли из видимого журнала разговора.

На более позднем скриншоте восстановленный разговор начинается с успешного Hello, после которого идёт моя явная заметка о том, что предыдущее сообщение исчезло [S18].



Один только скриншот состояния после восстановления не способен восстановить отсутствующую ветку. Последовательность выше основана на моём непосредственном наблюдении перехода.

Поэтому самая сильная формулировка намеренно узкая:

> **После следующей успешной отправки из видимого разговора исчезли и провалившийся ход, и его состояние ошибки.**

Это сильнее, чем просто сказать, что Retry не сработал. В видимом UI перестала существовать вся провалившаяся ветка.

Но это всё ещё не доказывает постоянное серверное удаление. Пропавшая ветка может быть следствием неудачной фиксации, сверки с более старым долговечным

состоянием, восстановления с устаревшей реплики, ошибки связывания вложений или другой границы сохранности.

Однако при работе с чёрным ящиком практическое правило простое:

> **Видимое — не значит долговечное. Retry — не значит восстановление.**

И отсюда следует второе правило:

> **Журнал разговора после сверки состояния может скрыть сам факт того, что сбой вообще происходил.**

Именно поэтому здесь важны скриншоты: они сохраняют

событие, которое поздний UI уже может не показывать.

21.5 Поздний улов: сообщение исчезло, файл остался

Этот улов появился уже после того, как рукопись была закончена.

24 сентября 2026 года я вернулся к разговору, куда накануне загрузил пятиминутную запись экрана для совсем другого анализа. В видимой истории сообщений вложения уже не было. Пропажу я заметил именно потому, что собирался дальше обсуждать эту запись.

Самое интересное началось после исчезновения. В отдельном файловом интерфейсе извлечения, доступном ассистенту, исходный артефакт по-прежнему находился под своим первоначальным именем:

```
ScreenRecording_09-23-2026 12-15-47_1.mp4
```

Размер файла составлял 213 074 795 байт. Его удалось снова извлечь и открыть как корректный MP4 длительностью около 302,77 секунды [E20].

Получилась аккуратная инверсия наблюдения о сроках

жизни артефактов из раздела 2.

Там состояние разговора могло сохранять знание об артефакте уже после исчезновения прямого доступа к его байтам.

Здесь видимый разговор потерял вложение, а сами байты артефакта остались доступными в другом месте.

Наблюдалось

- в пользовательском журнале загруженное видео больше не отображалось;
- отдельный файловый интерфейс всё ещё находил исходный артефакт;

- байты удалось снова извлечь и успешно декодировать;
- следовательно, отсутствие вложения в журнале не означало удаления самого артефакта.

Не установлено

Это **не** устанавливает, почему вложение исчезло из журнала, была ли запись сообщения удалена, скрыта, откачена или просто перестала отрисовываться, относится ли сохранившийся файл к той же системе долговременного состояния, что и разговор, как долго он ещё будет доступен и существует ли в обычном

клиентском UI независимый путь его восстановления.

Поэтому узкий вывод такой:

> **Присутствие в видимом журнале и доступность артефакта могут расходиться в обе стороны.**

А практическое правило ещё проще:

> **Нет в разговоре — не значит нет в хранилище.**

Полевое имя наблюдения:

> **Ягода не существует.**

22. Врезка: вторая предиктивная система ошиблась иначе

Во время подготовки доказательств для бага генерации изображений я заметил несвязанное предсказание клавиатуры iOS:

```
7 + 6 = 594
```

Арифметика выглядела абсурдно, пока соседние числа не раскрыли связную интерпретацию:

```
6 × 14 × 7 + 6 = 594
```

Вычисление правильное. Предположенная системой задача — нет.

Это полезный боковой пример того же более общего режима отказа: предиктивная система может оставаться локально согласованной, выбрав неправильный контекст.

Арифметика была правильной.
Рассуждение было локально последовательным.
Ответ был абсурдным.

Система генерации изображений сохранила неправильный визуальный контекст; клавиатура выбрала неправильный числовой

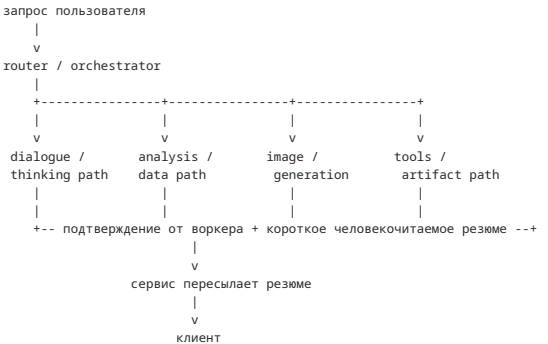
контекст. В обоих случаях дополнительный контекст не спас намерение. Он вытеснил его.

Избыточный контекст может быть не менее опасен, чем недостаточный.

Совпадение получилось уместным: доказательства, собранные для одного бага поведения модели, породили второй пример из другой предиктивной системы. Исследование начало генерировать собственные побочные квесты.

23. Возможная модель чёрного ящика

Ниже — одна из возможных архитектурных моделей, а не реконструкция, доказанная скриншотами. Эти блоки помогают организовать гипотезу «воркер / подтверждение / наблюдатель»; новые вечерние доказательства сами по себе эти блоки не идентифицируют.



Сам воркер может дополнительно разложить работу на части:

```
worker
  |
  +-> subtask A -> summary A
  +-> subtask B -> summary B
  +-> subtask C -> summary C
  |
  +-> aggregate / continue
```

Это могло бы объяснить, почему видимый статус меняет язык или даже содержит смешанные языковые фрагменты без участия языкового поведения пользователя.

Но это снова только рабочая гипотеза.

Возможен и более простой баг многоязычной генерации.

Путь отказа может выглядеть иначе:

```
worker
  |
  X crash / stop / lost
lease / abnormal
termination
  |
```

```
health observer /
supervisor
  |
machine outcome
  |
service maps outcome to
fixed English description
  |
client
```

Это соответствовало бы видимому контрасту между контекстным narration прогресса, переведённым при необходимости:

```
Analyzing statuses
Classifying run states
Reviewing diagram visuals
```

и сухими описаниями ошибок:

```
Stopped thinking
Stopped analyzing
Error analyzing
The request timed out.
```

Система также может сохранять работу по фазам:

```
сбор
  -> checkpoint

анализ
  -> checkpoint

создание артефакта
  -> checkpoint

финализация / доставка
```

Тогда поздний сбой оставлял бы более раннюю работу

доступной следующему выполнению.

Наконец, сохранность разговора, похоже, имеет собственную границу:

```
видимый ход клиента
    |
    v
долговечная фиксация
разговора
    |
    v
контекст, доступный
следующему run
```

Эпизод с откатом указывает, что эти стадии не обязаны быть одновременными.

Вся эта схема — инструмент рассуждения.

Реальная реализация может использовать очереди, RPC, акторы, движки рабочих процессов, общие воркеры, специализированные модели или нечто совсем иное.

Важное конкурирующее объяснение требует меньше придуманных компонентов: одна разговорная модель может выбирать инструменты, пока клиент показывает подписи инструментов, метаданные источников и сгенерированные резюме активности. Та же модель может выбрать не тот инструмент или неправильно понять метафору. Скриншоты этого не исключают.

Поэтому новые доказательства помогают различать роли представления и видимые действия пользователя. Но они не выбирают топологию воркеров, не доказывают очередь, не доказывают буквальную пересылку резюме и не устанавливают автоматические контрольные точки фаз.

24. Что мы действительно знаем

Наблюдения чище группируются по границам, чем по хронологии.

Артефакты и контекст

- Похожие на локальные пути /mnt/data/... могут появляться в UI, а текстовое знание об артефакте способно переживать прямой доступ к его байтам.
- Видимые в разговоре ссылки и временный доступ к файлам могут иметь разные сроки жизни; наблюдаемое поведение также указывает, что активный контекст не тождествен полной истории чата.
- Содержимое, вставленное с форматированием, может стать .webarchive-подобным артефактом и пройти через

видимые шаги обработки файлов перед использованием.

Статусы и представление

- Метки активности могут соответствовать смыслу задачи, переформулироваться и менять язык.
- Одна и та же область статуса может смешивать сохраняющиеся строки истории, меняющееся повествование, текст запроса, счётчики и иконки.
- Stopped thinking, Stopped analyzing, Stopped reasoning, Error analyzing и сообщения

о таймауте — разные видимые метки. Сами по себе они не идентифицируют состояния конкретных воркеров.

- Видимая кнопка Stop может сосуществовать с меткой остановки, а карточка ошибки может исчезнуть, пока ход пользователя остаётся.

Маршрутизация и выбор инструментов

- Короткие следующие сообщения после хода с изображением могут продолжать генерацию изображений даже после

смены предполагаемой темы.

- Явная коррекция Don't draw вернула наблюдавшийся обмен к обсуждению.
- UI сообщил о клонировании репозитория одновременно с веб-поиском; это совместимо с выбором подходящих инструментов, но не подтверждает, что операция действительно была выполнена.

Поведение генерации

- Уже завершённое изображение может позднее сопровождаться ошибкой генерации изображения.

- Референсное изображение породило воспроизводимый дефект сохранения головы/шеи при смене позы; явная инструкция полностью пересчитать анатомию его смягчила.
- Генерация может семантически дрейфовать ещё до появления видимого отказа по безопасности.

Сбои и восстановление

- Клиент может сообщить о прерванном потоке, остаться зависшим в ожидании завершения и восстановить финальное состояние после перезапуска приложения.

- Эпизод Stopped analyzing пережил перезапуск до явного Stop, после чего новый ход породил свежую активность.
- Запрос с медиа-вложениями может получить тайм-аут, оставив видимую оболочку и кнопку Retry, которая не даёт наблюдаемого восстановления.
- Запрошенный артефакт может стать доступным после попыток, выглядевших провалившимися, хотя запись не устанавливает, какая попытка его создала и использовалась ли какая-либо внутренняя

контрольная точка повторно.

Сохранность разговора

- Ранее видимые ходы могут исчезать во время сбоев и восстановления.
- В одной зафиксированной последовательности ход пользователя сохранился, а его представление тайм-аута/Retry исчезло.
- В другой наблюдаемой последовательности провалившийся ход с вложениями и его состояние ошибки исчезли после следующей успешной отправки.

- В более позднем эпизоде загруженное видео исчезло из видимого журнала, но исходный MP4 остался доступен через отдельный файловый интерфейс извлечения.
- В конце продукт явно показал ограничение уровня разговора: This conversation is too long to continue. Start a new chat and try again.

Управление чёрным ящиком

- Небольшие ограничения на естественном языке могут менять тип действия, а не только формулировку ответа.

- Пошаговые инструкции для документных задач, запрашивающие меньшие единицы работы и явные контрольные точки, дали промежуточные файлы и финальную сборку.
- По мере роста и мультимодальности разговора нестабильность становилась заметнее, но контролируемых измерений частоты сбоев относительно размера не проводилось.

25. Чего мы не знаем

Скриншоты **не** раскрывают внутреннюю архитектуру. В

частности, я не могу определить:

Топология среды выполнения

Использует ли система одну среду выполнения или несколько; специализированы ли воркеры; превращаются ли подзадачи в дочерние воркеры; на каком языке реализован тот или иной сервис; соответствуют ли видимые классы задач отдельным моделям, конфигурациям или только состояниям представления.

Контекст и долговременное состояние

Как собирается активный контекст; основано ли извлечение на векторах; где находится долговечная фиксация разговора; какие промежуточные фазы, если вообще какие-либо, сохраняются; отражают ли исчезающие ходы неудачную фиксацию, устаревшие реплики, расхождение кэшей, задержку снимка, ошибку связывания вложений или что-то ещё.

Файлы и инструменты

Что физически соответствует /mnt/data; есть ли у временных артефактов фиксированный TTL; означает ли позднее выжившее вложение отдельное хранилище артефактов, библиотечную копию или иной слой сохранности; было ли заявленное клонирование репозитория успешным; что именно считает отображаемое количество сайтов; выполнялись ли операции инструментов последовательно или параллельно.

Статусы и восстановление

Приходят ли резюме прогресса от воркеров, координатора или клиента; приходят ли сообщения о сбоях от отдельного наблюдателя; почему возник Stopped analyzing; использует ли повторное подключение повторную доставку, повторное получение, опрос или сверку состояния; что именно Retry пытался сделать в случаях таймаута.

Генерация изображений и безопасность

Точные стадии проверок безопасности; источник

семантического дрейфа; слой, отвечающий за дефект мотоциклетного референса; является ли продолжение генерации изображения следствием отдельного маршрутизатора, обычного следования инструкции, выбора инструмента или их сочетания.

Нагрузка из-за размера разговора

Какая величина вызвала финальный отказ conversation is too long; находится ли это ограничение рядом с реальной границей надёжности; были ли ранние сбои вызваны размером разговора, просто

коррелировали с ним или появлялись только при конкретных нагрузках.

Эти неизвестные — не слабость. Это граница эксперимента.

Исследование чёрного ящика становится полезным, когда хватает дисциплины остановиться у этой границы.

26. Полевое правило

Я начал это исследование, следуя за маленькими аномалиями.

Путь.

Зависшая кнопка Stop.

Строка статуса.

Событие памяти.

Поток, который признал, что прервался, но всё равно продолжил ждать.

Stopped analyzing рядом с кнопкой Stop, оставшейся на экране.

Retry без наблюдаемого восстановления.

Сгенерированное изображение, запомнившее неправильную шею.

А затем арифметическое автодополнение, технически

правильное для вопроса, которого никто не задавал.

По отдельности ни одна из этих улик мало что значила.

Вместе они показали рисунок:

> **Проблема не в оптимизации. Проблема начинается, когда выбранный по догадке короткий путь становится устойчивым.**

Или ещё короче:

> **Ошибочная догадка становится состоянием.**

Это стало полезным редакционным правилом для всей коллекции наблюдений.

Потому что в современных AI-системах самые интересные баги могут жить не в строке кода.

Они могут жить в моменте, когда система формирует разумную интерпретацию, сохраняет её и продолжает быть разумной ещё долго после того, как реальность ушла дальше.

26.1 Разговор сам стал стресс-тестом

Только ближе к концу стала очевидна форма эксперимента.

Специального бенчмарка не было. Один разговор просто накопил реальную работу:

```
текст
+ изображения и видео
+ загрузки и
сгенерированные медиа
+ извлечение данных и
использование инструментов
+ создание артефактов
+ повторные попытки и
провалившиеся ходы
+ долгоживущее состояние
разговора
```

По мере роста этого состояния разговор накопил более широкий набор видимых сбоев: устаревшую активность, прерванную доставку, остановленные фазы,

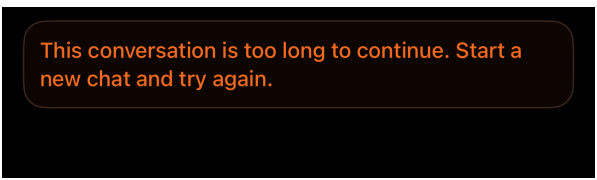
таймауты, неэффективные Retry, исчезающее состояние сбоя и исчезающую ветку сбоя.

Это **не** контролируемая кривая надёжности. Я не измерял частоту отказов относительно количества токенов, байтов вложений или числа вызовов инструментов. Наблюдение лишь в том, что нестабильность становилась заметнее по мере того, как разговор становился больше и мультимодальнее.

Затем продукт показал явную границу [S19]:

```
This conversation is too
long to continue.
```

```
Start a new chat and try
again.
```



Сообщение не раскрывает, какая величина достигла лимита, и не доказывает, что приближение к нему вызвало ранние сбои. Ограничение может быть задано на уровне продукта и вовсе не обязано совпадать с точкой, где начинает ухудшаться надёжность.

Именно здесь полезна аналогия со stack overflow, пока она остаётся аналогией:

```
комфортная область
    -> всё больше
пограничных эффектов
        -> неопределённый
запас
            -> явный
жёсткий предел
```

Разговор также задействовал разные видимые возможности: диалог, обработку файлов, генерацию изображений, retrieval, создание артефактов, восстановление и долговременное состояние. Это не доказывает существование отдельных бэкенд-сервисов. Но объясняет, почему один долгоживущий чат мог показать несколько разных границ отказа.

Разговор не проектировался как stress test. Он стал им путём накопления.

И эти сбои изменили то, как я с ним работал. Это последняя часть истории.

27. Улов, лежавший на виду

К концу я уже не только описывал ответ, который хотел получить. Я также ограничивал **то, как должна выполняться следующая часть работы**.

```
Не рисуй.
    -> вернуться к
```

```
обсуждению вместо ещё
одного изображения

Ты здесь? / Продолжай.
    -> попросить
продолжить, не повторяя всю
задачу

Работай маленькими этапами.
После каждого сохраняй
checkpoint.
    -> уменьшить размер
единицы работы и
материализовать
промежуточные артефакты
```

Это не документированные opcodes. Are you there? не было надёжной командой восстановления, а Don't draw — не безошибочный

переключатель. Аналогия с языком ассемблера полезна только на пользовательском уровне: короткая фраза может ограничить следующее действие, а не только стиль ответа.

Я называю это **инженерией маршрутизации**. То есть влиянием на то, будет ли система обсуждать, вызывать инструмент, продолжать прежнюю работу или явно материализовывать промежуточный результат. Это может объясняться обычным следованием инструкциям; может объясняться отдельным маршрутизатором; а может сочетанием обоих механизмов.

В этом пользовательском смысле поле разговора — ещё и управляющая поверхность на естественном языке.

Интересна именно хронология. Управляющие фразы появились **после** того, как сбои обнаружили слабые границы. Рабочий процесс с контрольными точками был не стилистической прихотью, а эксплуатационным обходным приёмом, выведенным из повторяющихся документных сбоев.

Та же адаптация появилась и в работе с изображениями. Когда

нежный промпт съезжал к более сексуализированной сцене, полезным ходом было не побеждать фильтр, а заранее точнее задавать желаемую несексуальную сцену. Со временем разговорный слой сам мог переводить общее намерение в более точный запрос генерации.

Так получается самый полезный цикл всего исследования:

Наблюдение -> архитектурная гипотеза -> эксплуатационная инструкция -> ещё один наблюдаемый результат.

Или проще:

> **Я не нашёл руководство к системе. Я вывел его части из сбоев.**

Это всё ещё не раскрывает скрытую архитектуру. Но раскрывает нечто более практичное: чёрный ящик способен научить пользоваться собой, не раскрывая внутреннее устройство.

Полезное управление. Несовершенная интерпретация. Неизвестная реализация.

Этого достаточно.

Не бойтесь случайно нарушить привычное поведение системы.

Нормальная работа склонна прятать структуру за рутиной. Неожиданный сбой, сломанное предположение или необычное состояние могут показать границы и связи, которые штатный путь (happy path) оставляет невидимыми.

> **Сбой — не катастрофа. Это приглашение исследовать.**

Иногда стоит спросить:

> **Что система только что раскрыла о себе?**

Там может оказаться крупная рыба.

Рыба поймана. Исследование закрыто. Перед подачей вынуть кости.

Приложение: полевые заметки из Музея просроченной еды

Несколько строк пережили исследование и заслуживают остаться рядом с доказательствами:

Результат получен. Метод получения не воспроизводится.

Работа завершена. Ожидание завершения продолжается.

Работа закончилась. Индикатору не сообщили.

Изображение было сохранено. Генерация изображения завершилась ошибкой.

Путь известен. Где файл — неизвестно.

Пользователь попросил объятие. Генератор импровизировал. Фильтр подал жалобу.

Просроченное не значит бесполезное. Некоторый контент всё ещё можно редактировать.

Retry был доступен. Восстановления не наблюдалось.

Ход сохранился. Состояние сбоя — нет.

Видимое — не значит долговечное. Retry — не значит восстановление.

Сообщение исчезло. Файл остался.

Я не нашёл руководство к системе. Я вывел его части из сбоев.

И одно методологическое правило:

> **Не путай хорошую историю об архитектуре с доказательством архитектуры.**

Это правило уже спасло исследование от нескольких очень привлекательных ошибок.

Приложение с доказательствами: выбранные кадры и полевые записи

Большинство записей ниже — вырезанные фрагменты предоставленных скриншотов, а не воссозданный UI. Порядок соответствует предоставленной

последовательности, а не восстановленному журналу событий. Более поздние кадры документируют исчезновение состояния таймаута, журнала после потери и явный отказ из-за длины разговора. E20 — короткая полевая запись о позднем эпизоде с сохранившимся вложением; в ней зафиксированы только факты, проверенные в той сессии.

ID	Доказательство

S01

Сохранённые подписи поиска/клонирования плюс поиск по конкретному домену

S02

Те же сохранённые строки плюс общая подпись поиска

S03

Те же строки плюс резюме финализации

S04

Подпись с количеством сайтов

S05

Представление поискового запроса

S06

Резюме задачи без видимой ведущей иконки

S07

Та же формулировка, теперь с зелёной иконкой

S08

Метка остановки, продолжение, видимый текст и последующая метка активности

S09

Вопрос о готовности и соответствующая подпись активности

S10

Подписи редактирования/упаковки

S11

Метка остановки, новое сообщение и затем возобновившаяся активность

S12

Последовательность после нового сообщения с более поздней подписью Shaping the conclusion

S13

Последовательность после нового сообщения с более поздней подписью Finalizing deliverables

S14

Общий статус Thinking из задачи перевода

S15 Короткое следующее сообщение, за которым следует Stopped reasoning
S16 Ход пользователя плюс Reasoning failed, карточка тайм-аута и Retry
S17 Тот же ход остаётся; тайм-аут / Retry исчезли; виден Stopped thinking

S18

Журнал разговора после восстановления, когда провалившаяся ветка уже исчезла

S19

Явный отказ уровня разговора: chat is too long to continue

E20

Поздний улов 24 сентября: вложение отсутствует в видимом журнале, но исходный MP4 остаётся извлекаемым

Включённый манифест доказательств содержит имена

файлов и их контрольные суммы SHA-256. Ошибка с аквариумом документируется запросом и сгенерированными результатами разговора; одни вечерние скриншоты не показывают всю эту последовательность.

Эти дополнения не дают бэкенд-логов, измерений памяти, идентичностей воркеров или контролируемого эксперимента. Их вклад — более ясная запись того, что было видно, чего просил пользователь и какое действие последовало.